 # Exercise 1: Spring Data JPA - Quick Example (Country Retrieval)

Scenario

This exercise demonstrates how to build a Spring Boot application using **Spring Data JPA and Hibernate** to retrieve data from a MySQL database table (country). The application follows a layered architecture using Entity, Repository, and Service classes.

Objective

The objective of this exercise is to understand how to:

- Configure Spring Boot with Spring Data JPA
- Connect Spring Boot application with MySQL database (XAMPP)
- Create and map entity classes using JPA annotations
- Use JpaRepository to perform database operations
- Implement service layer with transaction support
- Retrieve and display data from database using Spring Boot application

---

1. Understanding Spring Data JPA

Spring Data JPA simplifies database operations by:

- Eliminating boilerplate JDBC code
- Providing built-in CRUD operations
- Supporting repository-based data access
- Integrating Hibernate as ORM provider

It helps developers focus on business logic instead of SQL queries.

---

2. Dependencies

The following dependencies are used in pom.xml:

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

```
<dependency>
   <groupId>com.mysql</groupId>
   <artifactId>mysql-connector-j</artifactId>
   <scope>runtime</scope>
</dependency>

<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-devtools</artifactId>
</dependency>
```

---

3. Database Setup (MySQL - XAMPP)

Database Name:

ormlearn

Table Creation:

```
create table country(
   co_code varchar(2) primary key,
   co_name varchar(50)
);
```

Sample Data:

```
insert into country values ('IN', 'India');
insert into country values ('US', 'United States of America');
```

---

4. Implementation

4.1 Entity Class (Country.java)

This class maps Java object to database table.

@Entity

@Table(name = "country")

Fields:

- code → co_code (Primary Key)

- name → co_name

4.2 Repository Layer

public interface CountryRepository extends JpaRepository<Country, String>

This interface provides built-in methods such as:

- findAll()
- findById()
- save()
- delete()

4.3 Service Layer

The service layer contains business logic and uses repository methods.

```
@Transactional
public List<Country> getAllCountries() {
   return countryRepository.findAll();
}
```

4.4 Main Application (Execution Flow)

The application starts from main() method:

Steps:

1. Spring Boot application starts
2. ApplicationContext is created
3. CountryService bean is retrieved
4. getAllCountries() method is executed
5. Data is fetched from MySQL database

---

5. Output

When the application runs successfully, the following output is displayed:

Inside main

Start

Repository called

countries=[Country{code='IN', name='India'}, Country{code='US', name='United States of America'}]

End

---

6. Conclusion

This exercise demonstrates the successful implementation of **Spring Boot with Spring Data JPA** to perform database operations. It shows how a layered architecture (Entity–Repository–Service) simplifies database access and improves code maintainability. The application successfully retrieves data from a MySQL database using Hibernate ORM.

OUTPUT SCREENSHOT